

Project Interview Package

This document consolidates interview questions, concise model answers, a 2–3 minute demo script, and suggested improvements for the project.

Overview

Project: A lightweight React app (Vite) for browsing past exam questions (PYQs) and notes. Users can filter by branch/year, search, and view detailed content in a reading pane.

Target user: Students preparing for exams who need easy access to curated past questions and notes.

Interview Questions & Concise Answers

General

Q: What does this project do?

A: It provides a searchable, filterable viewer for past exam questions and notes with a two-column layout: filters/list on the left and detail view on the right.

Q: Who is the target user?

A: Students preparing for exams.

Tech Stack

Q: Which tools and libraries are used?

A: Vite, React (JSX), plain CSS. `package.json` and `vite.config.js` control build/dev.

Q: Why Vite over Create React App?

A: Faster dev server, modern tooling, smaller build config and faster HMR.

React Fundamentals

Q: How does state flow in the app?

A: Local state in parent containers (filters/selected item) is lifted and passed as props to child components; child events bubble up via callbacks.

Q: Which components are stateful vs presentational?

A: **Stateful:** main container holding filters and selected item. **Presentational:** `ContentList`, `ContentView`, `BranchFilter`, `SearchBar`.

Routing & Navigation

Q: How would you add multi-page routing?

A: Add `react-router-dom`, wrap app in `BrowserRouter`, define routes for home, item detail, legal pages; use lazy-loading for routes.

Components

Q: What does `BranchFilter` do?

A: Renders branch/year selectors and emits selection changes via callback.

Q: What does `ContentList` do?

A: Renders filtered items, handles empty state, and emits item selection events.

Q: What does `ContentView` do?

A: Shows selected item details including badges and formatted content.

Q: How would you test these components?

A: Use React Testing Library to render components with props, assert DOM content, and spy on callback invocations.

Data & APIs

Q: Where is data stored and how is it consumed?

A: Static module `data/pyqs.js` is currently imported and consumed at runtime; for scale, replace with API calls.

Q: How to handle async data and caching?

A: Use `useEffect+fetch` or `axios` and a caching library like React Query for retries and caching.

Styling

Q: What's the styling approach?

A: Global CSS in `src/styles.css` with CSS variables controlling theme and component classes.

Q: How to migrate to scoped styles?

A: Convert to CSS Modules, Tailwind, or styled-components for encapsulation.

Accessibility

Q: What accessibility improvements are needed?

A: Add semantic HTML, label inputs, keyboard focus management, ARIA attributes for dynamic content, and run axe checks.

Performance

Q: How to optimize render and bundle size?

A: Memoize components, use `useMemo/useCallback`, code-split routes, and remove large unused dependencies; profile with React DevTools.

Build & Deployment

Q: How to build and deploy?

A: `npm run build` (Vite) produces `dist/`. Deploy to Netlify, Vercel, GitHub Pages, or static host.

Testing

Q: What tests to add?
A: Unit tests for components (SearchBar, ContentList, ContentView), integration tests for filter→list flow, E2E for core user flows using Playwright or Cypress.

Security & Privacy

Q: Any sensitive data concerns?
A: Currently no sensitive data. For APIs, use HTTPS, server-side secrets, and avoid embedding keys in frontend.

Scaling & Architecture

Q: How would you change structure as app grows?
A: Add pages/, hooks/, services/, and ui/ folders; adopt TypeScript; use server state management (React Query) and split components into smaller primitives.

2–3 Minute Demo Script

- 1. Opening (15s)
 - "Hi, I'm [Your Name]. This is a lightweight React app I built to browse past exam questions and notes — users can filter by branch/year, search, and read full content."
- 2. Tech & architecture (25s)
 - "Built with Vite + React for fast dev and small bundles. UI uses a single global stylesheet (src/styles.css) and components live under src/components. Data is static for now in data/pyqs.js but the app is structured so it can fetch from an API."
- 3. Live walkthrough (60–90s)
 - "On the left are filters — BranchFilter controls branch/year selection and SearchBar filters by text. Selecting a branch or entering a query updates ContentList, which shows matching items. Click an item and ContentView displays details including badges and formatted content."
 - (Optional) "Key UI logic is in src/components/ContentList.jsx and src/components/ContentView.jsx."
- 4. Challenges & trade-offs (20s)
 - "I focused on readable layout and responsiveness. Trade-offs: I used static data for speed and simplicity; for scale I'd add server-side pagination and caching (React Query)."
- 5. Improvements & next steps (20s)
 - "Next: add API integration, unit + E2E tests, accessibility refinements (ARIA roles, focus management), and convert styles to CSS Modules or a component-scoped approach."
- 6. Closing / prompt (10s)
 - "That's the app — quick to scan and read study material. Would you like me to walk through the code for a specific component or show how I'd add API fetching and tests?"

High-Impact Improvements (short list)

- Convert to TypeScript.
- Add React Query for data fetching/caching.
- Add unit & E2E tests (Vitest/Jest + Playwright/Cypress).
- Convert styles to CSS Modules or Tailwind.
- Add CI (GitHub Actions) for lint/test/deploy.
- Add accessibility fixes and run automated a11y checks.

How to Convert This Markdown to PDF (two options)

Option A — Pandoc (recommended if you have it installed):

```
pandoc "docs/interview-package.md" -o "docs/interview-package.pdf" --pdf-engine=xelatex
```

Option B — Node + markdown-pdf via npx (runs without installing globally):

```
npx markdown-pdf "docs/interview-package.md" -o "docs/interview-package.pdf"
```

Notes:

- If a command fails due to missing tools, install pandoc or run the npx command which fetches required packages automatically.